

# The Cosmo42 Case Study

## 1. Introduction

The evolution of Large Language Models (LLMs) has led to unprecedented text comprehension and generation capabilities. However, base models suffer from two intrinsic limitations: information obsolescence (knowledge frozen at the training data cutoff) and the tendency to "hallucinate" (inventing plausible but false facts when the model does not know the answer). To overcome these limitations, the **RAG (Retrieval-Augmented Generation)** paradigm was created. Instead of relying exclusively on the LLM's parametric memory, a RAG system acts like an expert researcher: first, it searches for relevant information within a private and updated Knowledge Base (KB), and then it provides these documents to the LLM so it can formulate an answer based exclusively on the retrieved facts.

This approach guarantees transparency, source traceability, and the ability to update the system's knowledge simply by adding or removing files from the database, without needing to retrain the model.

### 1.1. RAG vs. Fine-Tuning

RAG and Fine-Tuning are often confused. The following table summarizes the key differences:

| Feature                  | RAG Systems                                           | Fine-Tuning                                      |
|--------------------------|-------------------------------------------------------|--------------------------------------------------|
| Data Update              | Instantaneous (just upload a new file)                | Slow (requires a new training cycle)             |
| Costs                    | Low (API and Vector DB costs)                         | Very High (GPU computational resources)          |
| Traceability             | Maximum (ability to cite the exact source)            | None (knowledge merges into the network weights) |
| Hallucination Prevention | High (the LLM is constrained to the provided context) | Low (the model can still invent facts)           |

## 2. The Cosmo42 Project: Overview

**Cosmo42** is an open-source enterprise platform developed by **exmachina**, a cutting-edge Swiss software house specializing in artificial intelligence. Released under the Java namespace `ch.exmachina.cosmo42`, the project aims to democratize access to multimodal RAG systems for businesses.

The core of Cosmo42 is a virtual assistant expert in retrieving information within a private Knowledge Base, asynchronously populated by users through the upload of various file formats (PDFs, text files, structured documentation).

Unlike traditional text-only RAG systems, Cosmo42 stands out due to its integration of a **Visual LLM**. This allows the system not only to understand extracted text but also to visually interpret complex layouts, charts, or diagrams present in the uploaded documents.

## 3. Processing Architecture and Search Flow

The beating heart of Cosmo42 is its indexing and search engine. When a user uploads a file, the system processes it by breaking it down into semantic "chunks" (fragments). Each chunk is then transformed into a mathematical representation (vector or embedding) and saved in a Vector Database.

When the user asks a question, the following **Mandatory Search Flow** is triggered:

1. **Key Concept Extraction:** The system does not forward the raw question (e.g., "tell me the results"). Instead, the LLM analyzes the user's intent and generates a descriptive natural language semantic search query (e.g., "Summary of results in the Q3 financial report").
2. **Similarity Search:** The generated query is vectorized and compared against the database using a specific tool (`search_knowledge_base`). The context "chunks" with the highest cosine similarity are retrieved.
3. **Analysis and Response:** The retrieved documents are passed to the LLM. If the documents contain the answer, the LLM processes the text professionally. If the documents are not relevant, the system is strictly instructed to declare: *"I have not found specific information in the documents at my disposal regarding this request"*, thus preventing any attempt at hallucination.

## 4. Technical Implementation

From a technological standpoint, Cosmo42 is developed in Java leveraging the **Spring AI** framework (specifically the 2.0.0-M6 milestone release). This choice ensures solid integration with the enterprise ecosystem.

### 4.1. Structured Outputs

To ensure that communications between the Java application and the LLM are robust, Cosmo42

heavily utilizes the *Structured Output* APIs. By using components like `ResponseFormat` in strict JSON Schema mode, the application forces the LLM to return data that maps perfectly onto predefined Java DTOs.

```
// Conceptual example of a DTO used in Cosmo42
public record KBSimilaritySearchRequest(
    @JsonPropertyDescription("A specific, context-rich search query written in natural language,
    designed to retrieve the most semantically relevant documents.")
    String query
) {}
```

## 4.2. Conversation Memory Management

A critical aspect of Cosmo42 is chat history management. To maintain state between HTTP requests in a scalable way, the system uses Spring AI's `JdbcChatMemoryRepository` abstraction.

To adapt to corporate database policies, the developers customized the implementation by bypassing the automatic schema initialization (`SPRING_AI_CHAT_MEMORY`) and extending the SQL dialects (e.g., `PostgresChatMemoryRepositoryDialect`) to point to custom persistence tables. The session is passed statelessly to the various Advisors via the request context using specific keys like `chat_memory_conversation_id`.

## 5. Transparency and Source Citation

The most important feature of an enterprise RAG system is reliability. The user must be able to trust the answer provided by the AI.

For this reason, Cosmo42's system instructions mandate maximum transparency through the use of UUIDs (Universally Unique Identifiers). Whenever Cosmo42 states a fact derived from the Knowledge Base, it is required to cite the exact source using a specific nomenclature.

If, for example, information is extracted from meeting minutes, the response will contain a tag like `REF_FILE_e58ed763-928c-4155-bee9-fdbaaadc15f3`. The application's front-end captures this tag and transforms it into a clickable link that takes the user to the exact page of the original uploaded document. The LLM undergoes strict checks to prevent the invention of non-existent UUIDs, ensuring that every reference points to a file mathematically retrieved from the vector database.